System Architecture (C4): RAG-Diary

> A three-level architecture model: Context, Container and Component, with the technology stack actually used and the architecture observations to close in the next round.

> Thai version: C4-ARCHITECTURE.md

1. Document control

Item	Detail
Document	System Architecture (C4)
Version	1.0
Prepared	15 October 2026
Author	Business Analyst / System Analyst (project owner)
Status	Reviewed, round 1 (25 October 2026)
Related documents	DATA-MODEL, SEQUENCE-DIAGRAMS, ADR, NFR-SPEC

2. Purpose and scope

This document answers one question: what is the system made of, which part talks to which, and where are the boundaries? It gives the reader the whole picture on one page before the detail.

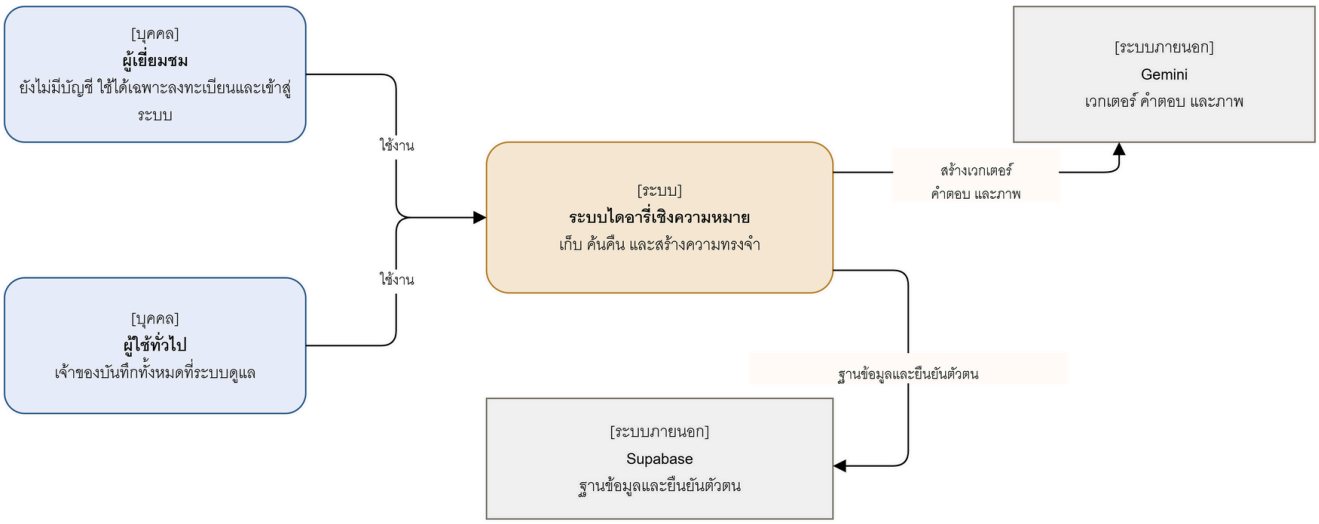
Scope: Context, Container and Component level architecture. Out of scope: the data model (in DATA-MODEL), detailed message sequence (in SEQUENCE-DIAGRAMS) and individual architecture decisions (in ADR).

3. Technology stack in use

Layer	Technology	Role
Frontend	Next.js App Router, Bun, React, TypeScript	Renders screens and manages client state
UI	Tailwind CSS, TanStack Query	Styling and cached data handling
Backend	Elysia, Bun, TypeScript	API and business logic

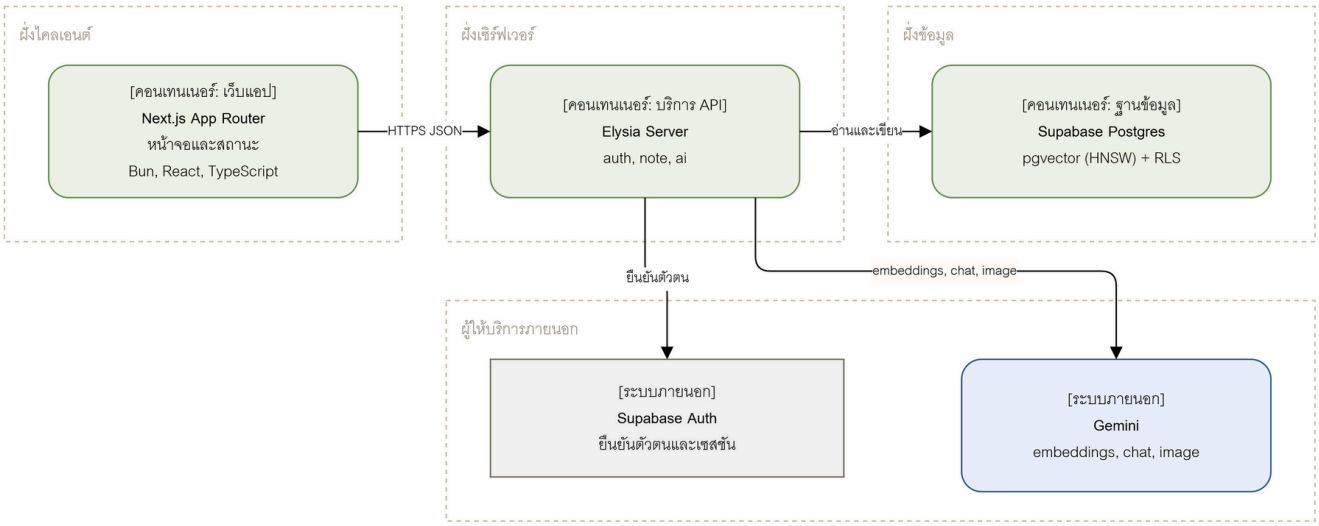
Layer	Technology	Role
Database	Supabase Postgres with pgvector (HNSW)	Stores data and the vector index
Security	Supabase row-level security	Isolates data per user
Identity	Supabase Auth	Authentication and session management
AI	Google Gemini (embeddings, chat, image)	Builds vectors, answers and images

4. Level 1: Context



- Visitor: has no account yet, and can only register and log in.
- Registered user: owns every entry the system manages.
- Google Gemini: provides embeddings, answers and image generation.
- Supabase: provides the database and authentication.

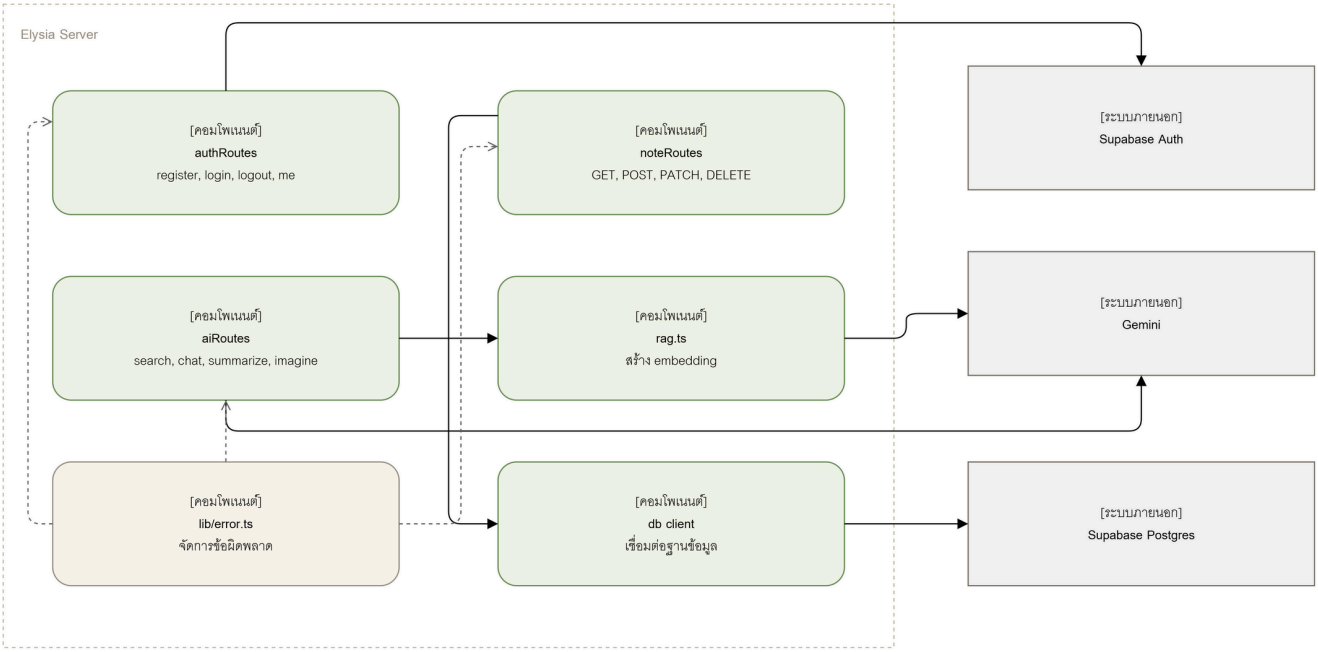
5. Level 2: Container



- Frontend: shows screens, manages the session and sends API requests.

- API server: all business logic.
- Database: stores entries and the vector index, and enforces isolation with RLS.
- Supabase Auth: authenticates users and issues sessions.
- Google Gemini: handles all three AI functions.

6. Level 3: Component



Component	File	Responsibility	Related use cases
authRoutes	modules/user/auth.ts	Register, log in, log out, check the session	UC-01
noteRoutes	modules/notes/note.ts	Create, read, edit and delete entries	UC-02, UC-03
aiRoutes	modules/ai/ai.ts	Semantic search, chat, mood summary and image generation	UC-04, UC-05
rag.ts	modules/ai/rag.ts	Build embeddings from entry text	UC-02
authGuard	lib/auth.ts	Read the session from the httpOnly cookie (with a Bearer token fallback) and bind a database client to the user's rights	Every UC that touches data
error.ts	lib/error.ts	Map errors into the API response shape	Every UC
db client	lib/db.ts	Connect to the database and run queries	Every UC that touches data

7. Architecture observations

Topic	Current state	Proposal
System shape	Modular monolith split by function	Keep it until there is a reason to split services
Vector storage	pgvector with an HNSW index	Suitable for the v1.0 scale
Data isolation	Enforced by RLS in the database	Always test isolation as its own case
Token storage	httpOnly cookies behind a same-origin proxy (ADR-10)	JavaScript cannot read them; every request must go through the Next.js proxy
Endpoint protection	Per-user rate limiting is active (search 60, chat 20, summarize 20, imagine 5, notes 60 per minute)	Verified 12 Nov 2026: the 6th /ai/imagine request returned 429 with Retry-After 60

8. References

Document	Relationship
DATA-MODEL	The structures the containers manage
SEQUENCE-DIAGRAMS	Message sequence between components
ADR	Why each technology was chosen
NFR-SPEC	The measures the architecture must support